

Apostila Java — Enums, Controle de Fluxo e Boas Práticas com Scanner

Jader Silva · [GitHub](#) · [LinkedIn](#)

Sumário

1. `System.exit(0)`
 2. Métodos que recebem Scanner como parâmetro
 3. Introdução aos Enums
 4. Enum com construtor e métodos
 5. Particularidades do Enum
 6. Enum com métodos abstratos
 7. Enum implementando interface
 8. Switch com Enum
 9. Dicas profissionais
-

1. `System.exit(0)`

`System.exit(int status)` encerra a JVM (Java Virtual Machine) imediatamente, interrompendo a execução do programa naquele exato ponto — nenhuma linha de código depois dela é executada.

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Início");  
        System.exit(0);  
        System.out.println("Essa linha nunca será impressa");  
    }  
}
```

O que significa o parâmetro (0, 1, etc.)

O valor passado é o **código de saída** (*exit code*) do processo, usado pelo sistema operacional ou por scripts que chamam o programa:

- 0 → convenção universal para "terminou com sucesso"

- Qualquer valor diferente de `0` (ex: `1`, `2`) → convenção para "terminou com erro" — o número pode até ser usado para identificar *qual* erro ocorreu, em scripts de automação

```
if (saldo < 0) {  
    System.out.println("Erro: saldo negativo detectado.");  
    System.exit(1); // sinaliza erro para quem executou o programa  
}
```

Onde costuma aparecer

- Menus de console, na opção "Sair":

```
switch (opcao) {  
    case 5:  
        System.out.println("Encerrando o sistema...");  
        System.exit(0);  
        break;  
}
```

- Validações críticas, onde não faz sentido o programa continuar (ex: arquivo de configuração não encontrado).

Cuidado

`System.exit()` é uma saída **abrupta**: ela não executa blocos `finally` pendentes de forma garantida em todos os cenários, não fecha recursos automaticamente (como `Scanner`, conexões de banco, arquivos abertos) a menos que você feche antes, e não deve ser usada dentro de métodos de bibliotecas ou APIs — só em pontos de decisão do próprio `main` /menu principal. Em código de produção com testes automatizados (JUnit), `System.exit()` dentro do código testado costuma ser evitado, porque encerraria também o processo dos testes.

2. Métodos que recebem Scanner como parâmetro

Um padrão muito comum (você já usou isso: `lerInformacao(Scanner sc, String msg)`) é passar o `Scanner` **como parâmetro** de métodos auxiliares, em vez de criar uma nova instância dentro de cada método.

```

public static String lerTexto(Scanner sc, String msg) {
    System.out.print(msg);
    return sc.nextLine();
}

public static int lerInteiro(Scanner sc, String msg) {
    System.out.print(msg);
    return sc.nextInt();
}

```

Por que reaproveitar a mesma instância

`Scanner` envolve um recurso do sistema (`System.in` , a entrada padrão). Se você criar múltiplas instâncias (`new Scanner(System.in)` várias vezes) apontando para o mesmo `System.in` , pode causar comportamento inconsistente de buffer — um `Scanner` pode "roubar" dados que o outro esperava ler. A prática correta é:

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in); // única instância

        String nome = lerTexto(sc, "Nome: ");
        int idade = lerInteiro(sc, "Idade: ");

        sc.close(); // fecha só no final, depois de todo uso
    }
}

```

Passando Scanner entre classes

O mesmo princípio vale quando o `Scanner` precisa ser usado em métodos de **outras classes** (ex: uma classe de menu chamando um método de cadastro):

```

public class Cadastro {
    public void executar(Scanner sc) {
        System.out.print("Digite o nome: ");
        String nome = sc.nextLine();
        // ...
    }
}

```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        Cadastro cadastro = new Cadastro();  
        cadastro.executar(sc); // reaproveita a mesma instância  
        sc.close();  
    }  
}
```

Isso evita duplicar `Scanner` s pela aplicação e centraliza o fechamento do recurso em um único lugar (normalmente o `main`).

3. Introdução aos Enums

`enum` (abreviação de *enumeration*) é um tipo especial usado para representar um **conjunto fixo e conhecido de constantes**. É a ferramenta certa sempre que uma variável só pode assumir um número limitado de valores possíveis — dias da semana, status de pedido, naipes de baralho, categorias de conta bancária, etc.

Sintaxe básica

```
public enum DiaDaSemana {  
    SEGUNDA, TERCA, QUARTA, QUINTA, SEXTA, SABADO, DOMINGO;  
}
```

Cada nome dentro do `enum` é uma **constante** — na prática, uma instância única e pré-criada do próprio tipo `DiaDaSemana` .

Usando o enum

```
DiaDaSemana hoje = DiaDaSemana.QUARTA;  
  
if (hoje == DiaDaSemana.SABADO || hoje == DiaDaSemana.DOMINGO) {  
    System.out.println("Fim de semana!");  
} else {  
    System.out.println("Dia útil.");  
}
```

Por que usar enum em vez de `String` ou números "mágicos"

Um erro comum de quem está começando é representar categorias fixas assim:

```
// Abordagem frágil – evite
String status = "ATIVO"; // e se alguém digitar "ativo", "Ativo ", "ATIV00"?
int categoria = 2; // o que significa "2"? Sem contexto, é ilegível.
```

Com `enum`, o compilador garante que só valores válidos existem — não tem como criar um `DiaDaSemana.SEXTAA` por engano, o código nem compila. Isso elimina uma classe inteira de bugs de digitação/comparação de texto e torna o código autoexplicativo.

4. Enum com construtor e métodos

Um `enum` em Java **não é só uma lista de nomes** — é uma classe especial, e pode ter atributos, construtor e métodos, exatamente como uma classe comum.

```
public enum Planeta {
    MERCURIO(3.3e23, 2.4e6),
    VENUS(4.87e24, 6.05e6),
    TERRA(5.97e24, 6.37e6);

    private final double massa;    // em kg
    private final double raio;     // em metros

    // construtor do enum
    Planeta(double massa, double raio) {
        this.massa = massa;
        this.raio = raio;
    }

    public double calcularGravidade() {
        final double G = 6.67300E-11;
        return G * massa / (raio * raio);
    }
}
```

Regras do construtor de enum

- O construtor é **sempre implicitamente private** (ou pode ser omitido, mas nunca `public` / `protected`) — você não pode instanciar um enum manualmente com `new`, então não faz sentido um construtor acessível de fora.
- Os valores passados entre parênteses na declaração das constantes (`MERCURIO(3.3e23, 2.4e6)`) são os argumentos desse construtor.
- Cada constante do enum pode ter valores diferentes para os mesmos atributos.

Usando

```
Planeta terra = Planeta.TERRA;  
System.out.printf("Gravidade na Terra: %.2f m/s²%n", terra.calcularGravidade());
```

Esse padrão é extremamente útil sempre que cada constante carrega **dados próprios** associados a ela — não só um nome, mas um comportamento ou valor específico.

5. Particularidades do Enum

5.1 — Comparação com `==`

Enums podem (e **devem**) ser comparados com `==`, ao contrário de `String` (onde a boa prática é `.equals()`):

```
if (dia == DiaDaSemana.SABADO) { ... } // forma correta e segura
```

Isso é seguro porque **cada constante de enum é um único objeto na JVM** — não existem duas instâncias diferentes de `DiaDaSemana.SABADO` coexistindo. O `==` compara referência de objeto, e como só existe uma referência por constante, funciona perfeitamente (e é mais eficiente que `.equals()`).

5.2 — Não se instancia com `new`

```
DiaDaSemana d = new DiaDaSemana(); // ERRO DE COMPILAÇÃO
```

O Java proíbe isso propositalmente. As instâncias de um `enum` são criadas **uma única vez**, automaticamente, quando a classe é carregada — e ficam disponíveis como constantes estáticas (`DiaDaSemana.SEGUNDA`, etc.). Isso é o que garante a segurança do `==` explicada acima.

5.3 — `values()`

Método gerado automaticamente pelo compilador para **todo** enum. Retorna um array com todas as constantes, na ordem em que foram declaradas:

```
for (DiaDaSemana dia : DiaDaSemana.values()) {  
    System.out.println(dia);  
}  
// SEGUNDA  
// TERÇA  
// QUARTA  
// ...
```

Muito usado para percorrer todas as opções possíveis (ex: popular um menu, validar entrada).

5.4 — `valueOf()`

Também gerado automaticamente. Converte uma `String` na constante correspondente do enum:

```
DiaDaSemana dia = DiaDaSemana.valueOf("QUARTA");
```

Atenção: o texto precisa bater **exatamente** (incluindo maiúsculas/minúsculas) com o nome da constante, senão lança `IllegalArgumentException`. É comum combinar com `try/catch` ao ler entrada do usuário:

```
try {  
    DiaDaSemana dia = DiaDaSemana.valueOf(sc.nextLine().toUpperCase());  
} catch (IllegalArgumentException e) {  
    System.out.println("Dia inválido!");  
}
```

5.5 — `ordinal()` e `name()`

```
DiaDaSemana d = DiaDaSemana.QUARTA;  
System.out.println(d.ordinal()); // 2 (posição na declaração, começando em 0)  
System.out.println(d.name());    // "QUARTA" (nome exato da constante)
```

`ordinal()` é geralmente evitado como lógica de negócio (se você reordenar as constantes no código, os números mudam silenciosamente e podem quebrar regras que dependiam da ordem antiga). Prefira comparar pela própria constante (`==`) em vez de por posição numérica.

5.6 — Enums podem implementar interfaces

Veja a seção 7, dedicada a isso.

5.7 — Enums não podem estender (`extends`) outras classes

Todo `enum` já herda implicitamente de `java.lang.Enum` — e Java não permite herança múltipla de classes. Por isso, um `enum` pode **implementar interfaces**, mas nunca **estender** outra classe.

5.8 — Enum dentro de `switch`

Veja a seção 8.

6. Enum com métodos abstratos

Um `enum` pode declarar um **método abstrato**, obrigando **cada constante** a fornecer sua própria implementação — é polimorfismo aplicado dentro do próprio `enum`.

```
public enum Operacao {
    SOMA {
        @Override
        public double calcular(double a, double b) {
            return a + b;
        }
    },
    SUBTRACAO {
        @Override
        public double calcular(double a, double b) {
            return a - b;
        }
    },
    MULTIPLICACAO {
        @Override
        public double calcular(double a, double b) {
            return a * b;
        }
    };

    public abstract double calcular(double a, double b);
}
```

Usando

```
double resultado = Operacao.SOMA.calcular(5, 3); // 8.0
System.out.println(resultado);

for (Operacao op : Operacao.values()) {
    System.out.println(op + " -> " + op.calcular(10, 2));
}
```


Por que isso é útil

Sem método abstrato, você provavelmente cairia num `switch` gigante fora do enum:

```
// Abordagem menos elegante
public double calcular(Operacao op, double a, double b) {
    switch (op) {
        case SOMA: return a + b;
        case SUBTRACAO: return a - b;
        case MULTIPLICACAO: return a * b;
        default: throw new IllegalArgumentException();
    }
}
```

O problema dessa segunda abordagem: toda vez que uma nova operação é adicionada ao enum, alguém precisa lembrar de voltar nesse `switch` e adicionar um novo `case` — se esquecer, o bug é silencioso. Com método abstrato, o compilador **obriga** você a implementar `calcular()` em toda nova constante assim que ela é criada — é impossível esquecer, porque o código não compila sem isso.

7. Enum implementando interface

Como mencionado na seção 5, um `enum` pode implementar uma ou mais interfaces normalmente — o que é útil para dar a ele um contrato comum com outras classes do sistema.

```
public interface Tributavel {
    double calcularImposto(double valor);
}

public enum CategoriaProduto implements Tributavel {
    ALIMENTO(0.05),
    ELETRONICO(0.18),
    VESTUARIO(0.12);

    private final double aliquota;

    CategoriaProduto(double aliquota) {
        this.aliquota = aliquota;
    }

    @Override
    public double calcularImposto(double valor) {
        return valor * aliquota;
    }
}
```

Isso permite que um `enum` seja tratado de forma polimórfica junto com classes comuns que implementam a mesma interface:

```
public static void imprimirImposto(Tributavel item, double valor) {
    System.out.printf("Imposto: %.2f%n", item.calcularImposto(valor));
}

imprimirImposto(CategoriaProduto.ELETRONICO, 1000); // funciona normalmente
```

8. Switch com Enum

O `switch` tem uma sintaxe simplificada quando usado com `enum`: **não é necessário** prefixar o nome da classe em cada `case`.

```
DiaDaSemana dia = DiaDaSemana.SABADO;

switch (dia) {
    case SEGUNDA:
    case TERCA:
    case QUARTA:
    case QUINTA:
    case SEXTA:
        System.out.println("Dia útil");
        break;
    case SABADO:
    case DOMINGO:
        System.out.println("Fim de semana");
        break;
}
```

Repare: é `case SABADO`, **não** `case DiaDaSemana.SABADO` — o compilador já sabe o tipo pela variável testada no `switch`, e escrever o nome completo geraria erro de compilação.

Versão moderna (switch expression, Java 14+)

```
String tipo = switch (dia) {
    case SEGUNDA, TERCA, QUARTA, QUINTA, SEXTA -> "Dia útil";
    case SABADO, DOMINGO -> "Fim de semana";
};
```

Além de mais compacta, essa versão é **exaustiva**: se você esquecer de tratar alguma constante do enum, o compilador acusa erro (ao contrário do `switch` tradicional, onde esquecer um `case` só vira bug em tempo de execução).

9. Dicas profissionais

1. **Enum é a ferramenta certa sempre que o conjunto de valores é fixo e conhecido em tempo de compilação.** Se a lista de opções vem de um banco de dados ou pode crescer dinamicamente, geralmente não é caso de enum.
2. **Nomeie constantes de enum em MAIÚSCULAS**, seguindo a convenção Java para constantes (`ATIVO` , `INATIVO` , não `Ativo` , `ativo`).
3. **Sempre trate `IllegalArgumentException` ao usar `valueOf()` com entrada do usuário** — é a forma mais comum desse método falhar em produção.
4. **Prefira comparar enums com `==`**, nunca com `.equals()` desnecessariamente (embora `.equals()` funcione também, por herdar de `Object` , `==` é mais idiomático e igualmente seguro para enums).
5. **Evite basear lógica de negócio em `ordinal()`** . Se alguém reordenar as constantes ou inserir uma no meio, todo código que dependia da posição numérica quebra silenciosamente.
6. **Um enum pode ter métodos "privados" e atributos auxiliares**, exatamente como uma classe — não se limite a campos simples, ele pode encapsular lógica complexa quando fizer sentido.
7. **`System.exit()` deve ser usado com moderação** — normalmente só em pontos de saída claros do programa (menu principal, validação crítica de inicialização). Evite espalhar chamadas a ele pelo meio de métodos de regra de negócio.
8. **Sempre feche o `Scanner` no final do `main` (`sc.close()`)**, especialmente quando ele foi passado como parâmetro para vários métodos — centralize o fechamento em um único lugar para não correr o risco de fechar cedo demais e quebrar uma chamada posterior.
9. **Enum com método abstrato é preferível a `switch` externo** sempre que a lógica de "o que fazer" pertence naturalmente a cada constante — o compilador passa a garantir que nenhuma constante fique sem implementação.
10. **Um enum pode ter um construtor com múltiplos parâmetros**, e nada impede constantes de um mesmo enum terem lógicas internas complementares (ex: um método que usa dois atributos diferentes para cada constante).